MÉMO

# Court-circuit : le mémo

Ton robot colorie les cases où il passe ; une case adverse sur laquelle tu passes devient la tienne. Après 30 s, le plus de cases gagne.

Prénom : ..... Classe : .....

En ligne : **babash.github.io/court-circuit**

À VOS SMARTPHONES !

Jouer

## Les actions (une à la fois)

| | |
|---|---|
| `nord()` `sud()` `est()` `ouest()` | avance d'une case dans cette direction et la colorie ; le nord est en haut de ton écran |
| `eclair()` | éclair dans les 4 directions, sur 3 cases : l'adversaire touché est figé 2 s et les cases autour de lui deviennent les tiennes |
| `poserCaisse()` | pose une caisse sur ta case (3 par match) : quand tu en sors, plus personne n'y passe |
| `attendre()` | ne fait rien |

## Les capteurs (des questions)

| | |
|---|---|
| `obstacleNord()` | Vrai si la case au nord est un mur, une caisse, le bord ou l'autre robot ; aussi `obstacleSud()`, `obstacleEst()`, `obstacleOuest()` |
| `couleurNord()` | `"vide"`, `"moi"`, `"adversaire"` ou `"mur"` ; aussi `couleurSud()`… |
| `cible()` | Vrai si un éclair toucherait l'adversaire ; dans une seule direction : `cibleNord()`, `cibleEst()`… |
| `distance()` `caisses()` | les cases jusqu'à l'adversaire ; les caisses qu'il te reste |

## L'essentiel

**Boucle bornée** : répéter un nombre de fois fixé.

```
for i in range(4):
    nord()
```

**Boucle non bornée** : répéter tant qu'une condition est vraie.

```
while True:
    while not obstacleNord():
        nord()
    est()
```

**Fonction** : donner un nom à des lignes, puis l'appeler.

```
def zigzag():
    nord()
    est()
zigzag()        # fait nord(), puis est()
```

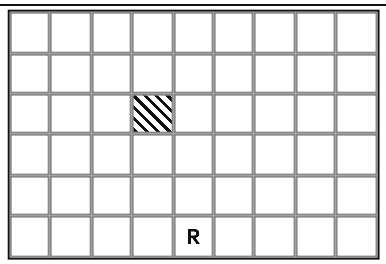
**Attention** : une ligne qui commence par `for`, `while`, `if`, `elif`, `else` ou `def` finit par « : », et les lignes de son bloc sont décalées de 4 espaces.

**Vitesse** : chaque ligne recopiée ralentit ton robot. Une boucle ou une fonction évite de recopier.

## Exercice 1 — Lire un code

Le robot part de la case R ; le nord est en haut. Colorier les cases qu'il visite avec ce code, puis compter les cases coloriées.

```
for i in range(3):
    nord()
for i in range(2):
    est()
```




Nombre de cases (départ compris) : .....

## Exercice 2 — Recopier ou répéter ?

Écrire avec une boucle `for` un code qui fait la même chose que : `nord()` `nord()` `nord()` `est()` `nord()` `nord()` `nord()` `est()`.

## Exercice 3 — Choisir sa condition

Compléter pour que le robot monte vers le nord sur les cases qui ne sont pas à lui, et aille à l'est sinon :

```
while True:
    if not obstacleNord() …… couleurNord() …… "moi":
        nord()
    else:
        ……
```

## Je sais…

- ☐ écrire une boucle `for` pour répéter un nombre de fois fixé ;
- ☐ poser une question au robot avec un capteur, comme `obstacleNord()` ;
- ☐ utiliser `while` pour répéter tant qu'une condition est vraie ;
- ☐ écrire `if` / `elif` / `else` avec les deux-points et le décalage ;
- ☐ définir une fonction avec `def` et l'appeler par son nom ;
- ☐ combiner deux conditions avec `and`, `or` ou `not`, et comparer avec `==`, `!=`, `<=`.

Court-circuit · Babash - NSI Tani Malandi · licence CC BY-SA 4.0 (documents) et MIT (code) · Code source sur GitHub : github.com/babash/court-circuit